



Anil Seth

Exploring Software—Free Shared Libraries!

It is time that package-management applications free shared libraries from artificial shackles, and create a more dynamic and versatile distribution.

The first time I came across UNIX shared libraries was with UNIX SVR3.2. The ability to work with multiple versions of libraries concurrently seemed impressive; as I recollect, the shared libraries we were using on mainframes did not offer such capabilities.

After about a year of using Linux, I found that I required that capability: we were using the Slackware distribution, and needed to install an additional package. We could get a binary version; however, it needed a lower version of a shared library. It was fairly simple to install the alternate version of the library as well, and get on with the work.

Normally, if a shared library is well-designed and well-maintained, an application developed or linked using a lower version should not have any problem working with a higher version of the library. The developer has to ensure that the old interfaces continue to be supported in later versions. If the library changes substantially, and the old interfaces no longer work, it may even be better to change its name—for example, *libxml2*, *libX11*, *libv4l1* or *libv4l2*.

Over time, repositories maintained by various distributions have made it far easier to transparently deal with library dependencies. We can still follow the same manual approach as above, but it is a barrier. GUI desktop packages, in particular, have much more complex dependencies than the server packages which we dealt with in what seems like a long time ago.

Distributions have simplified our lives, by imposing constraints that limit the flexibility of the underlying shared libraries. These constraints require that only one version of a library will be managed by the package-management software. Sometimes that is just not possible; in such a case, additional packages are created, typically prefixed with ‘compat’.

In the last few years, I have become convinced that this approach is unduly restrictive. As an example, upgrading to Fedora 13 created needless minor failures and friction because of library versions. In case anyone is interested in details, see my blog post, <http://sethanil.blogspot.com/2010/06/migrating-to-fedora-13.html>. If the version of a library installed from an F12 repository is higher than the version available in the F13 repository, why should it break an upgrade?

Another example occurred with Arch Linux. Since new Xorg and Intel display drivers are unstable on some older hardware, I have installed Arch Linux with the old Intel driver, and am holding back the Xorg packages to the versions that work with that driver. Recent updates succeeded; however, X stopped running. Xorg needed *libcrypt.so.0.9.8*, while the rest of the system needed the new *libcrypt.so.1.0.0*. In this case, as expected, the shared library did not change the interfaces—so

existing packages could continue to use the new version. The solution was simple: just create a symbolic link to the current version, whose name includes the desired version of the library. However, suppose Xorg is not updated, the existing library is still needed. Smarter package management would leave the existing *libcrypt* in place, and install the higher version as well.

It is a standard practice in package management to separate shared libraries from executable applications. This is because of an implicit assumption that libraries are different. Yet, though library packages need to be treated differently—they aren't. For example, as we discussed last month, the Debian/Ubuntu *apt* system keeps track of how a package was installed—manually, or to satisfy a dependency. In the latter case, when no one needs this package, it can be removed. Libraries are always used by some package, and will normally be installed to satisfy dependencies. Unfortunately, complications can arise as some libraries may be optional, and may be explicitly (manually) installed. An example that readily comes to mind is the *libdvdcss* library.

However, just as multiple versions of the kernel can be installed, a package manager should allow installation of multiple versions of library packages. Further, if a library package is installed to satisfy a dependency, then, as in the case of *apt*, it should remove this package once no other package needs it. Since there could be unusual problems, it is perfectly reasonable for a distribution to offer a warning like the one Firefox gives for *about:config*: “This may void your warranty!” The distribution can disown responsibility for the chaos that may result if multiple versions of a library are installed—but it shouldn't deny us that pleasure!

Once library packages are differentiated, package management can make use of that information. There are already a huge number of packages to choose from, and library packages just add to the noise. A simple estimate: on my system, 657 out of 2575 installed packages have ‘lib’ in their names. I neither need, nor want, to see the library packages in the package manager. Hiding library packages from view, by default, would definitely simplify package management. Shared libraries should normally work silently behind the scenes.

Moral: Avoiding a constraint that allows for more complex installation scenarios can result in increased simplicity! 

About Anil Seth

The author works as a consultant. Prior to consulting, Anil was a professor at Padre Conceicao College of Engineering (PCCCE) in Goa, managed IT and imaging solutions for Phil Corporation Limited (Goa), supported domestic customers for Tata Burroughs/TIL, and was a researcher with IIT-K and the Indian Institute of Geomagnetism (Mumbai). You can find him online at <http://sethanil.com> and reach him via e-mail at anil@sethanil.com